



Google Summer of Code

ML4SCI



GSoC 2025

Quantum Kolmogorov-Arnold Networks for High
Energy Physics Analysis at HEP

Project Proposal

By

Dilshad Raza

Table of Content

1. About Me
 - 1.1 Student Information
 - 1.2 University Information
2. Background Work and Programming Skills
3. Evaluation Task
4. Project Information
 - 4.1 Project: Quantum Kolmogorov Arnold Network for High Energy Physics Analysis at LHC
 - 4.2 Abstract
 - 4.3 Ideas
5. Approach
 - 5.1. Ansatz Construction:
 - 5.1.1 Using Sequential Application and LCU for taking Linear Combination
 - 5.1.2 Using Hadamard product for linear combination:
 - 5.2. Training regime:
 - 5.3 High-level flow diagram of training loop
 - 5.4 High-level flow diagram of Ansatz (Method-I)
6. Timeline (Tentative):
 - 6.1 Pre-GSoC period
 - 6.2 Timeline
 - 6.3 End-Term evaluation
7. Deliverables
8. Future Scope
9. Motivation for GSoC
10. Achievements
11. References:

About Me

Student Information

Name:	Dilshad Raza
Email:	dilshadraza010@gmail.com 23je0574@gmail.com
Timezone:	New Delhi, India (+5:30 GMT)
Github:	malevolentshryne
LinkedIn:	Dilshad Raza
Resume:	resume
Kaggle:	Kaggle Profile
Address:	Budhiakhad, Giridih, Jharkhand - 815301

University Information

University:	Indian Institute of Technology (ISM) Dhanbad
Major:	Mathematics and Computing
Current Year:	2 nd
Expected Graduation date:	August 2028
Degree:	Integrated M.Tech

Background Work and Programming Skills

- I am a second year undergraduate student at IIT (ISM) Dhanbad, pursuing Integrated M.Tech in Mathematics and Computing.
- I use Google Colaboratory, Jupyter and Kaggle notebooks for implementing Python code, and Microsoft Visual Studio code for development.
- I am proficient in Python, C++, Pytorch, PennyLane and Qiskit. I am also comfortable using git and github.
- I possess a fervent dedication to machine learning research, backed by valuable expertise in research methodology and data analysis.
- My interests include theoretical deep learning with an emphasis on Expressiveness of deep learning models and analysing their representational power theoretically.
- I am an active participant in Kaggle Competitions to apply my theoretical knowledge in real life problems. Currently I'm participating in the competition [Stanford RNA 3D folding](#).

Evaluation Task

I have successfully completed all the mandatory tasks outlined on the website, adhering to the provided guidelines and have uploaded them to [github](#).

In task-IX, I trained two classical KAN architectures.

First, Efficient-KAN which is basically an efficient implementation of vanilla B-spline KAN^[1] with a simplified regularization loss. Dropping the entropy loss and imposing L1 on weights only. This allowed us to skip the implicit loss calculation due to weights further optimizing the implementation.

Second, I trained BSRBF-KAN^[2] which has shown to achieve the state-of-the-art results for MNIST dataset among the KAN family, and was verified as well. Combination of RBF and B-spline works better than standalone B-spline because RBF can do better interpolation in scattered data meaning the data which is not structured on a grid. Their structure allows easy adaptation for complex non-linear approximations.

The quantum extension of the classical KAN is discussed later in detail.

Project Information

Organization: ML4SCI

Project: Quantum Kolmogorov Arnold Network for High Energy Physics Analysis at LHC

Possible Mentors:

- **Eric Reinhardt** (University of Alabama)
- **Dinesh Ramakrishnan** (University of Alabama)
- **KC Kong** (University of Kansas)

Project Length: 175 hours

Kolmogorov-Arnold Networks (KANs) have the potential to replace MLPs. The key difference between them is that while MLPs use node weights as vectors of scalar elements, KANs use vectors containing univariate nonlinear functions. This extra layer of non-linearity is what makes KAN so powerful. According to the Kolmogorov Arnold representation theorem, any multivariate continuous function can be approximated by a two-layer KAN with $(2n+1)$ nodes in the single hidden layer. This differs from the Universal Approximation Theorem(UAT) as UAT doesn't comment on the number of neurons required for a 2-layer network to learn a multivariate function. Although some approximations suggest that it requires $O(1/\epsilon^2)$ nodes for sigmoid activation functions, where ϵ denotes the approximation error([Barron's Theorem](#)). The KAN extends this two-layer idea to a L-layer MLP like structure.

$$\text{KAN}(\mathbf{x}) = (\Phi_{L-1} \circ \Phi_{L-2} \circ \cdots \circ \Phi_1 \circ \Phi_0)\mathbf{x}.$$

Quantum Kolmogorov-Arnold Networks (QKANs) extend KANs into the quantum domain, exploiting quantum superposition and entanglement to represent complex functions with a shallower architecture than classical networks. This project focuses on developing a QKAN architecture with the PennyLane framework and using it for High Energy Physics Analysis at LHC.

Abstract

As data complexity increases, traditional computing may become insufficient, making quantum computing a potential alternative. Using the PennyLane framework, we aim to develop a working Quantum Kolmogorov-Arnold Network architecture. This project focuses on developing and applying QKAN on various tasks related to High Energy Physics at LHC.

Ideas

I'll be working this summer on developing a Variational block-encoding based QKAN architecture, so the whole project will revolve around that

- **Developing the Block-encoding based ansatz:** First, I'll work on implementing the parameterized ansatz provided in the paper^[3].
- **Using Hadamard product:** Using Hadamard for computation of linear combination instead of sequential application.
- **Trying different optimizers:** Since gradient calculation using the parameter-shift rule may not be effective here, we need to explore alternative optimization techniques, such as the finite difference method.
- **Dataset Preparation and Preprocessing Procedure:** I will experiment with various physics-based preprocessing techniques that can be applied to the given dataset.
- **Searching for the best loss function:** Choosing a suitable loss function (or constructing a custom loss function, if required) for the specific tasks involving specific datasets.
- **Possibility of parameterizing other than weights:** As discussed in the paper^[3], parameterizing within QSVT can enhance the expressibility of the Ansatz.
- **Implementing different basis functions using QSVT:** If time permits, I will explore implementing different basis functions, such as B-splines, Fourier transforms. However, this may require a complete reconstruction of the ansatz and parameterization.

I believe the relevant skills required are Python, PyTorch, PennyLane, and a solid understanding of deep learning techniques. Additionally, familiarity with linear algebra concepts is also essential.

Approach

Quantum Chebyshev KAN trained with Variational frameworks:

1. Ansatz Construction:

Using Sequential Application and LCU for taking Linear Combination

The ansatz proposed in this project is derived from the one that was suggested in a thoroughly researched and mathematically based paper “QKAN: Quantum Kolmogorov-Arnold Networks”^[3]. The selection of this ansatz fits the overall objective of the project, which is to utilize a theoretically robust QKAN architecture for High Energy Physics (HEP) datasets. Instead of concentrating on developing a new quantum architecture, theory development on a large scale, I have focused more on constructing a complete and working QKAN model based on existing foundations.

I propose implementing the Quantum Chebyshev KAN and optimizing it using Variational Quantum Algorithms. The core idea of this architecture revolves around block encodings and QSVT.

I will begin by explaining block encoding and QSVT, as it forms the foundation for all subsequent formulations.

Block Encodings: Block encoding is a technique used to encode a matrix into another matrix (a unitary matrix), in an extended Hilbert space. Since quantum computers naturally work on unitary matrices hence this technique is very useful. We encode everything—from the input and Chebyshev polynomials to the weights—using block encodings. Block encoding of Chebyshev polynomials is obtained using QSVT and more particularly through reflection operators.

QSVT and Chebyshev Polynomial realization: We'll use Quantum Singular Value Transformation(QSVT) for constructing Chebyshev Polynomials in the form of block encodings. This realization of Chebyshev Polynomials using QSVT is near-optimal, and we can also try to replicate any other function which are polynomially approxable.

The classic SVD(Singular Value Decomposition) states that any matrix A can be decomposed into its singular values:

$$A = U\Sigma V^\dagger$$

Where Σ denotes the matrix with singular values in its diagonals. QSVT extends this idea by enabling direct polynomial transformations of singular values, allowing us to compute polynomial functions of X using QSVT.

$$A' = UP(\Sigma)V^\dagger$$

Mathematically, Chebyshev Polynomials of first kind are defined as:

$$\begin{aligned} T_0(x) &= 1, \\ T_1(x) &= x, \\ T_{n+1}(x) &= 2x T_n(x) - T_{n-1}(x) \end{aligned}$$

Now we can realize this in the circuit using reflection operators. Reflection operators are used to construct controlled phase rotations that allow us to “steer” the polynomial that the quantum circuit is implementing. We alternately apply U_x and $U_x^\dagger$ interleaved by reflection operators, to mimic the recursive relation of Chebyshev polynomials.

$$A = R \cdot U_x^\dagger \cdot R \cdot U_x$$

Now in our case, the reflection operator reflects the ancilla register about the $|0\rangle$ state to enforce the polynomial structure within a quantum circuit.

$$Z_\pi := I \otimes (2|0\rangle\langle 0|_{\text{ax}} - I_{\text{ax}})$$

To construct the r_{th} Chebyshev polynomial $T_r(x)$, we repeat this process $(r/2)$ times (assuming r is even). This repetition acts as a quantum analogue of recursive polynomial construction, and enables the circuit to encode the transformation $T_r(x)$ into the amplitude of the quantum state.

This results in a block encoding construction of the r^{th} term of the Chebyshev polynomials U_{Tr} , we repeat this for $(d + 1)$ times to get separate block encodings of $(d + 1)$ terms.

Once the Chebyshev polynomial terms $T_r(x)$ are constructed via separate block-encodings, the next step is to combine them with learned weights $w_r \in \mathbb{R}^{N_K}$, one for each degree $r \in [0, d]$. These weights are represented as diagonal matrices, and we assume access to corresponding block-encodings U_{wr} for each.

To multiply each weight with its corresponding polynomial term, I employ the technique described in Proposition 3 of the referenced paper, which allows for multiplication of two block-encoded operators via sequential application. Specifically, given:

- A block-encoding U_{Tr} of the Chebyshev operator $T_r(x)$,
- And a block-encoding U_{wr} of the weight diagonal matrix,

the product $w_r \cdot T_r(x)$ is implemented via:

$$(I \otimes U_{Tr})(I \otimes U_{wr})$$

This yields a new block-encoding for the composite operator, with controllable ancilla overhead and additive error.

This modular composition step is repeated for all $r \in [0, d]$, producing weighted block-encodings of all Chebyshev terms, which are subsequently combined using LCU.

The LCU is implemented by introducing $\log_2(d + 1)$ ancilla qubits and preparing an equal superposition over the $(d + 1)$ terms, followed by the controlled application of the corresponding U_r unitary blocks. We denote the obtained block encoding by $U(\phi)$.

After obtaining a block-encoding $U(\phi)$ of the diagonal matrix representing $N * K$ individual activations $\phi_{pq}(x_p)$, I apply Hadamard gates to the n input qubits on both sides of the block-encoding. This Hadamard transform aggregates the input activations for each output node in superposition, effectively realizing the operation:

$$U_{\text{output}} = (H_n \otimes I_k) \cdot U_\phi \cdot (H_n \otimes I_k)$$

The final outcome of the CHEB-QKAN construction is a unitary block-encoding $U(\phi)$ that maps an input vector of dimension N to an output vector of dimension K . This unitary, and its controlled form, can be used as the input to a subsequent layer, enabling recursive composition of multiple layers similar to deep classical neural networks but not the same.

Using Hadamard product for linear combination:

In the above sequence of paragraphs, I've discussed how we can obtain the linear combination of Chebyshev polynomials by multiplication of block-encoded operators via sequential application. One other way of doing this is through the Hadamard product. Hadamard product between a parameterized quantum circuit (PQC) and the prepared diagonal block-encoding of a Chebyshev polynomial can also do the same thing.

We consider the Hadamard product:

$$U(\boldsymbol{\theta}) \circ \langle 0 |_{\text{aux}} U_{\text{Tr}}^{(r)} | 0 \rangle_{\text{aux}}$$

Since U_{Tr} is diagonal, it contains only entries along the diagonal — its Hadamard product with any matrix will retain only diagonal entries. Hence, the Hadamard product with the PQC gives a new diagonal matrix, where each entry is:

$$\left[U(\boldsymbol{\theta}) \circ U_{\text{Tr}}^{(r)} \right]_{ii} = U(\boldsymbol{\theta})_{ii} \cdot T_r(x_i)$$

This approach can benefit because of the following reasons:

- In this method, the parameters θ directly control the diagonal entries of the matrix.
- These entries can range freely within $[-1,1]$, providing high expressibility.
- This Hadamard-PQC method imposes only an l_∞ constraint which is less restrictive than l_2 and allows the model to represent a wider function class.

2. Training regime:

I'll optimize QKAN parameters based on variational methods, where parameters of the ansatz are updated in an iterative manner to reduce a cost function. As the standard parameter-shift rule does not apply to my situation, I will approximate gradients with the finite difference method. Since the output of the quantum model is a diagonal block-encoding of a real-valued vector, the loss can be computed by measuring the output amplitudes and comparing them to ground truth.

Loss Function: First I'll choose an adequate loss function. The choice of our loss function would depend on the task and dataset. Let the chosen loss function be:

$$\mathcal{C}(\boldsymbol{\theta}) = \frac{1}{M} \sum_{i=1}^M \mathcal{L} \left(\hat{y}^{(i)}(\boldsymbol{\theta}), y^{(i)} \right)$$

In case of our model, the observable can be denoted by:

$$\mathcal{C}(\boldsymbol{\theta}) = \frac{1}{M} \sum_{i=1}^M \mathcal{L} \left(\langle \psi^{(i)}(\boldsymbol{\theta}) | \mathcal{O} | \psi^{(i)}(\boldsymbol{\theta}) \rangle, y^{(i)} \right)$$

More generally, model predictions are computed as expectation values of Hermitian observables:

$$\langle \mathcal{O} \rangle_{\boldsymbol{\theta}, \mathbf{x}} = \langle 0 | U_{\Phi}^{\dagger}(\boldsymbol{\theta}, \mathbf{x}) \mathcal{O} U_{\Phi}(\boldsymbol{\theta}, \mathbf{x}) | 0 \rangle$$

where $\langle \mathcal{O} \rangle_{\boldsymbol{\theta}, \mathbf{x}}$ can be decomposed into Pauli basis operators for efficient measurement. These values are then fed into a cost function $\mathcal{C}(\boldsymbol{\theta})$, which is minimized using classical gradient-based optimization.

Gradient Estimation via Finite Difference Method: To optimize the parameterized quantum circuit, I will use the central finite difference approximation to estimate the gradient of the cost function $\mathcal{C}(\theta)$ with respect to the parameters θ . The gradient is computed as:

$$\frac{\partial \mathcal{C}(\theta)}{\partial \theta_i} \approx \frac{\mathcal{C}(\theta + \epsilon e_i) - \mathcal{C}(\theta - \epsilon e_i)}{2\epsilon}$$

Where e_i is a unit vector in the direction of θ_i , and ϵ is a perturbation parameter.

This method gives a first-order approximation to the gradient and is easy to perform without modification of quantum circuits.

Once the gradients are calculated, I'll use the classical optimization algorithms like gradient descent and adam optimizer to achieve convergence.

Possible issues I may face and the proposed solutions:

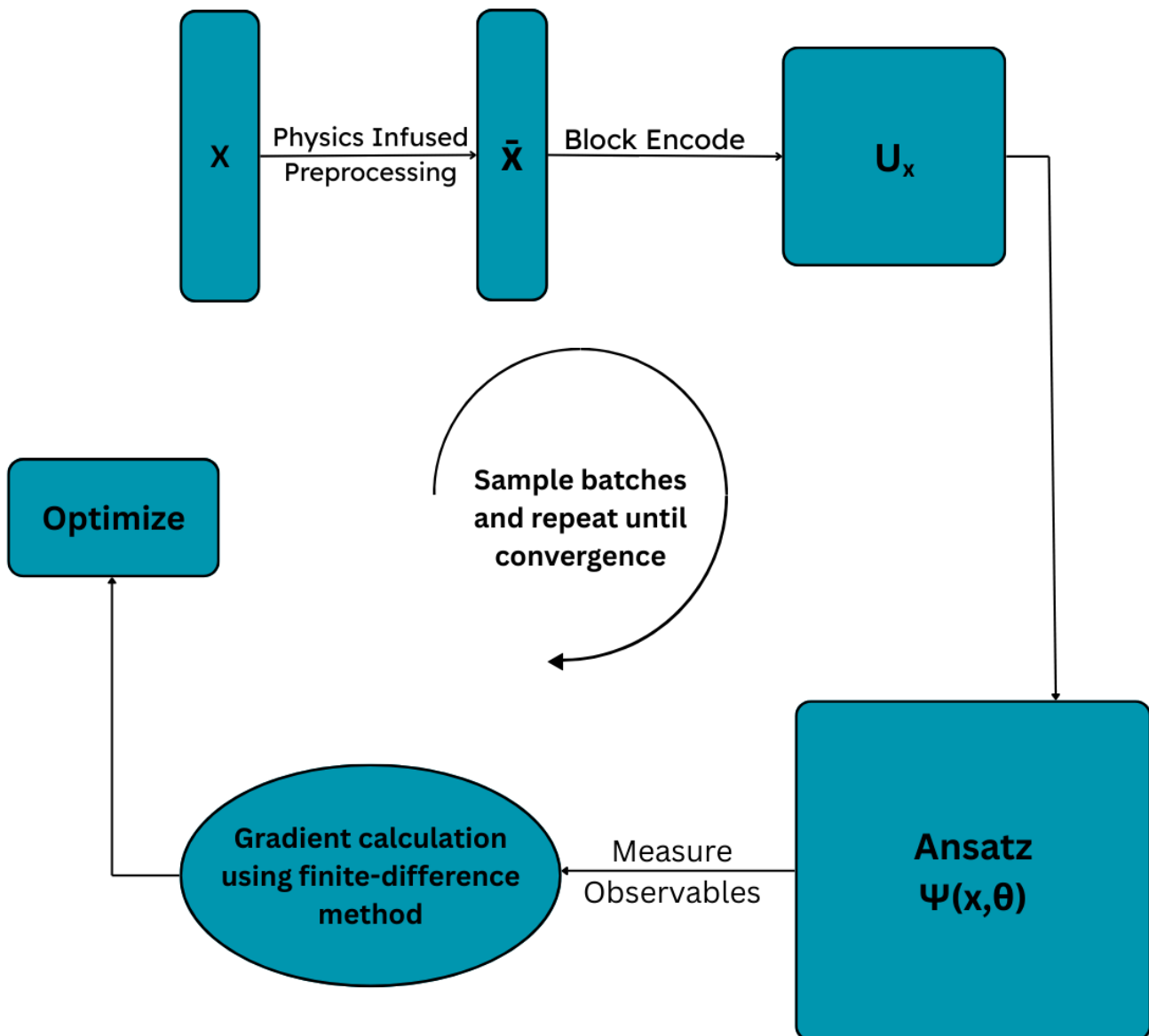
- Small perturbations can cause numerical instability. I will have to tune carefully.
- Barren Plateaus: To counter the problem of vanishing gradients in deep quantum circuits, I will try initialization schemes like layer-wise pretraining or parameter rescaling.

I'll explore Quantum Natural Gradient (QNG) descent as a potential alternative. QNG updates parameters using the inverse of the quantum Fisher information matrix, which naturally adapts to the underlying geometry of the quantum state manifold.

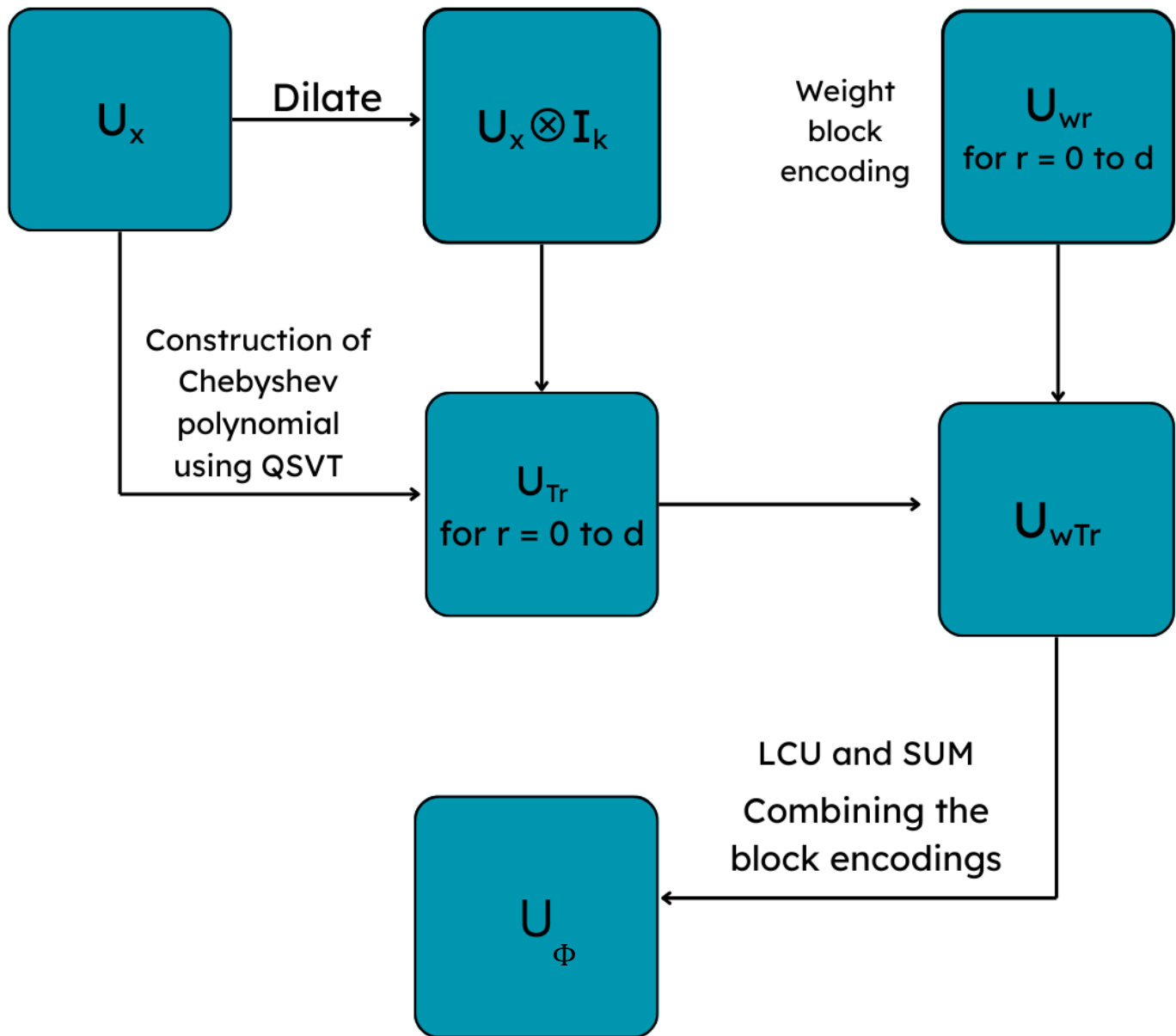
$$\theta \leftarrow \theta - \eta F_Q^{-1} \nabla_{\theta} \mathcal{C}(\theta)$$

This gives a more accurate calculation of gradient and fast convergence, and it is also robust to barren plateaus. The downside of this method is its computational inefficiency. As estimating each element of F_Q requires sampling and averaging. Also, calculating the inverse classically costs $O(n^3)$ time.

High-level flow diagram of training loop



High-level flow diagram of Ansatz (Method-I)



Timeline (Tentative):

Total Project Length:-175 hours

From my past experiences, I can say that things can change in a minimal time. For example, some bug comes up with a high priority level that needs to be fixed before any work. However, having a timeline is always suitable for any project. This, at least, helps you in keeping a tab on how much you are lagging.

Pre-GSoC period

Before the coding phase begins at the end of May, I'll be using this period to deepen my understanding of Variational Quantum Algorithms (VQAs) and Quantum Singular Value Transformation (QSVT). I'll focus on exploring their latest developments, especially those relevant to layered quantum architectures and Chebyshev polynomial-based constructions. Additionally, I'll get comfortable implementing these ideas in PennyLane through focused experiments and prototype circuits, ensuring I'm fully prepared to build and optimize complex quantum models during the main GSoC timeline.

The plan for GSoC 2025 will be:

I am planning to split the project into four phases:

- Phase 1: Preparing the first parameterized ansatz and training with variational optimization.
- Phase 2: Building the second QKAN using Hadamard product for multiplication and comparing the two.
- Phase 3: Hyperparameter tuning, benchmarking on the given datasets and tasks.
- Phase 4: Project Setup Cleanup and Documentation. Help new contributors grasp quickly and contribute.

Timeline

Period	Task
Week 1 [2 June - 8 June] Week 2 [9 June - 15 June]	• Study all the related research papers. • Prepare the environment and required tools. • Study the related physics involved to understand the dataset. • Process and clean the dataset. • Start preparing the codebase. • Benchmarking the classical KAN on specified tasks and keeping it in my codebase.
Week 3 [16 June - 22 June] Week 4 [23 June - 29 June] Week 5 [30 June - 6 July]	• Develop the first parameterized Ansatz. • Selection of a suitable loss function. • Training the first QKAN by the first method (Multiplication by sequential application and LCU) • Resolving the barren plateau problem (If observed) • Trying different optimizers including quantum natural gradient descent (QNG) and analyzing their computational efficiency.
Week 6 [7 July - 13 July]	• Deciding on the choice of loss function, gradient calculation, and optimizer. • Working on the second method (multiplying using Hermitian product).

Mid Term Evaluation

[14 July - 18 July]

Week 7 [14 July - 20 July]	• Training the QKAN using the second method. • Benchmarking the two approaches on the specified tasks.
Week 8 [21 July - 27 July] Week 9 [28 July - 3 August] Week 10 [4 August - 10 August]	1. Studying the effects of the following parameters • Number of layers (L) • Number of hidden layer nodes. • Number of Chebyshev polynomial terms (d) • Documenting the experimentations. • Choosing the best set of parameters. 2. Parameterizing the QSVT and analyzing its expressivity. 3. Experimenting with different post-processing techniques.
Week 11 [11 August - 17 August]	• Final benchmarking with the best model found. • Wrap up the code base and documentation.
Week 12 [18 August - 24 August]	• Write reports. • Buffer time to remove any blockers. • Any further enhancements.

End-Term evaluation

Goals: Wrap up

By this time up, I will ensure that all of the above implementations have been documented and tested. I will extend this period to my future work by writing blogs explaining all the works and if my work leads to a research paper, I will want to be a part of it.

I will update my progress weekly to my mentors and incorporate their feedback and suggestions as and when required. I am confident enough to complete the project on time. I have also kept a buffer week before final evaluations for any critical blockers.

Deliverables

- Most accurate and computationally efficient Quantum Kolmogorov Arnold Network to be used for a variety of regression and classification tasks.
- Fully implemented with the PennyLane framework.
- Experimentations with different loss functions, optimizers, and variational settings.
- Experimentations with number of Chebyshev terms $(d + 1)$ and expressivity against it.
- Benchmark on the given high energy physics datasets and comparison against classical KAN.
- Optional support for Quantum Natural Gradient (QNG) based updates.
- A Github Repository for all the above implementations.
- Beginner-friendly readme for easy installation with detailed steps.

Future Scope

This project will lead to a completely working Quantum Kolmogorov–Arnold Network, serving as a foundation for hybrid quantum-KAN architectures that leverage the expressiveness of quantum circuits. It will open new directions in quantum machine learning for modeling complex functional relationships, especially in high-dimensional scientific datasets. Integrating quantum computing into practical machine learning pipelines, has potential implications for high energy physics, quantum chemistry, and beyond.

My work can be extended to:

- Developing other Quantum KAN architectures by replacing Chebyshev polynomials as basis function. We can realize other basis functions using QSVT.
- Investigating layer-wise training and quantum skip-connections to mitigate exponential gate complexity growth.
- Theoretically analyze the expressivity of QKAN vs. classical KAN using polynomial approximation theory.

Motivation for GSoC

I'm a dedicated programmer and problem-solver, deeply passionate about coding and machine learning and am a member of the [Cyberlabs](#) machine learning division. I find joy in understanding algorithms that enable models to learn patterns and evolve through self-learning processes as well as I have a keen interest in mathematics and quantum computing and how machine learning is at the intersection of the two.

Contributing to open-source projects is not just a hobby but a fulfilling endeavor where I've had the opportunity to collaborate with diverse communities, honing both my technical skills and teamwork abilities. The prospect of creating solutions that could positively impact millions of lives fills me with immense pride, and I'm constantly motivated to expand my knowledge by delving into research papers and staying updated with the latest advancements in technology.

Achievements

- First Runner up, Winter of Code (WOC 6.0), 3 months long machine learning hackathon by IIT(ISM), Dhanbad.
- Second Runner up, Quanturing 2.0, 36 hours long Quantum Computing hackathon by IIT(ISM), Dhanbad.
- All India Rank 71 in Inter-Collegiate Programming Contest(ICPC) Amritapuri Regionals 2024-25.

THANK YOU !!

References:

1. [KAN: Kolmogorov-Arnold Networks](#)
2. [BSRBF-KAN: A combination of B-splines and Radial Basis Functions in Kolmogorov-Arnold Networks](#)
3. [QKAN: Quantum Kolmogorov-Arnold Networks](#)